WEEK-1

EXP NO. : 1A

Aim : Load VA Deaths(Death Rates in Virginia)dataset in R and visualize the data using different histograms.data("VADeaths")

Explanation:

- The primary purpose of data("VADeaths") is to load a built-in dataset into your R environment so you can work with it immediately.
- This specific dataset provides death rates per 1,000 in Virginia in 1940, cross-classified by age group and population demographic (Rural/Urban and Male/Female).
- To visualize this data using histograms in a grid, you first need to convert the matrix into a data frame and then use the hist() function for each column

Program :

```
print("VADeaths Dataset:")
print(VADeaths)
va_df <- as.data.frame(VADeaths)
print(head(va_df))
par(mfrow = c(2, 2))
```

Output :

	Rural Male	Rural Female	Urban Male	Urban Female
50-54	11.7	8.7	15.4	8.4
55-59	18.1	11.7	24.3	13.6
60-64	26.9	20.3	37.0	19.3
65-69	41.0	30.9	54.6	35.1
70-74	66.0	54.3	71.1	50.0

Program :

```
va_df <- as.data.frame(VADeaths)
print(head(va_df))
```

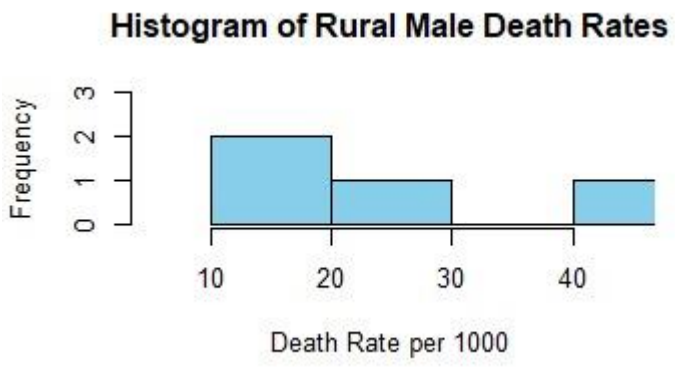
Output :

	Rural Male	Rural Female	Urban Male	Urban Female
50-54	11.7	8.7	15.4	8.4
55-59	18.1	11.7	24.3	13.6
60-64	26.9	20.3	37.0	19.3
65-69	41.0	30.9	54.6	35.1
70-74	66.0	54.3	71.1	50.0

Program :

```
par(mfrow = c(2, 2))
# Histogram for 'Rural Male' death rates
hist(va_df$`Rural Male`,
main = "Histogram of Rural Male Death Rates",
xlab = "Death Rate per 1000",
col = "skyblue",
border = "black",
xlim = c(5, 45), # Set consistent x-axis limits for comparison
ylim = c(0, 3))  # Set consistent y-axis limits
```

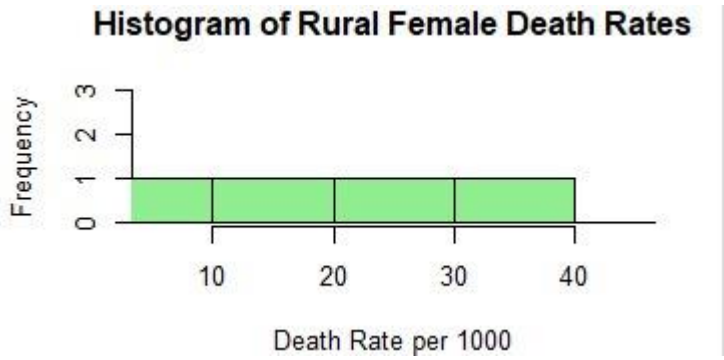
Output :



Program :

```
# Histogram for 'Rural Female' death rates
hist(va_df$`Rural Female`,
     main = "Histogram of Rural Female Death Rates",
     xlab = "Death Rate per 1000",
     col = "lightgreen",
     border = "black",
     xlim = c(5, 45),
     ylim = c(0, 3))
```

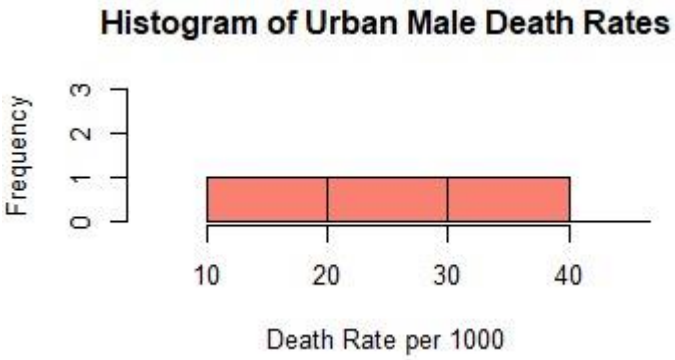
Output :



Program :

```
# Histogram for 'Urban Male' death rates
hist(va_df$`Urban Male`,
     main = "Histogram of Urban Male Death Rates",
     xlab = "Death Rate per 1000",
     col = "salmon",
     border = "black",
     xlim = c(5, 45),
     ylim = c(0, 3))
```

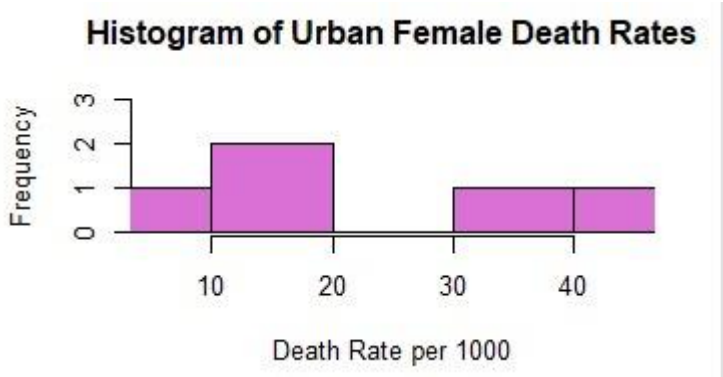
Output :



Program :

```
# Histogram for 'Urban Female' death rates
hist(va_df$`Urban Female`,
    main = "Histogram of Urban Female Death Rates",
    xlab = "Death Rate per 1000",
    col = "orchid",
    border = "black",
    xlim = c(5, 45),
    ylim = c(0, 3))
```

Output :

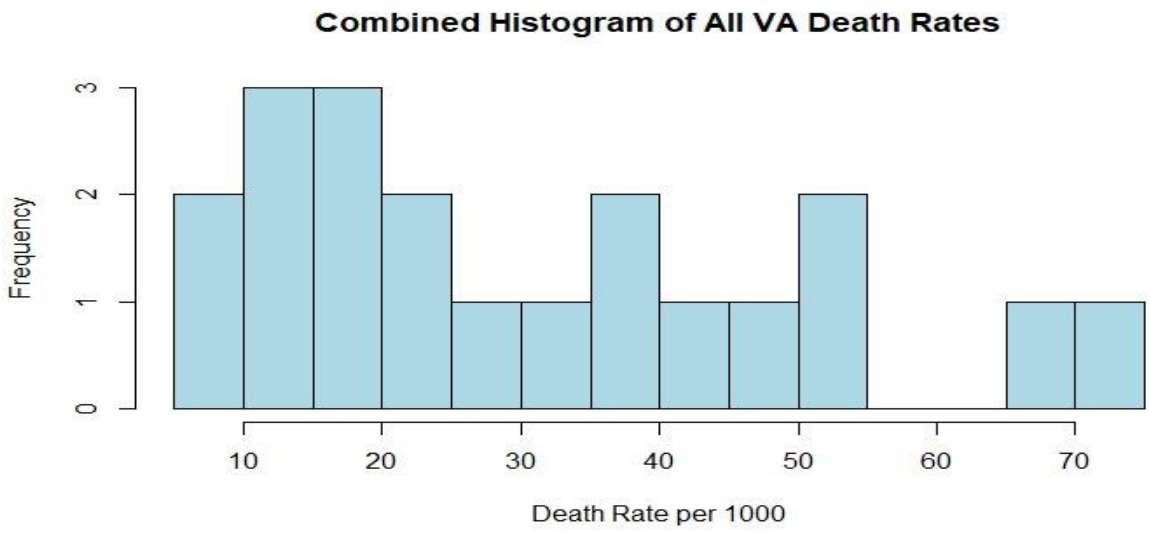


Program :

```
# Reset the plotting layout to
default (1 plot per window) after visualization
par(mfrow = c(1, 1))
# This requires reshaping the data first (code from source)
```

```
va_df$AgeGroup <- rownames(va_df)
va_long <- reshape(va_df, varying = list(1:4), v.names = "DeathRate",
                   times = colnames(va_df)[1:4], timevar = "Group",
                   direction = "long")
rownames(va_long) <- NULL

hist(va_long$DeathRate,
     main = "Combined Histogram of All VA Death Rates",
     xlab = "Death Rate per 1000",
     col = "lightblue",
     border = "black",
     breaks = 10) # Using more breaks for a detailed view
```

Output :



EXP NO. : 1B

Aim : Load air quality dataset in R and visualize La Guardia Airport's dialy maximum temperature using histogram.

Explanation:

- Airquality dataset and creating a histogram of the Temp variable is to perform Exploratory Data Analysis (EDA). Specifically, it allows you to:
- **See the Distribution:** You can instantly see how many days were cool (50s–60s) versus how many were hot (80s–90s).
- **Identify the "Normal" Weather:** The tallest bar (the mode) shows the most frequent temperature range at La Guardia during that period.
- **Detect Patterns:** It reveals if the temperatures are "skewed" (e.g., if there were significantly more heatwaves than cold snaps).
- **Spot Outliers:** You can easily identify extreme weather days that sit far away from the rest of the data.

Program :

```
data("airquality")
head(airquality)
```

Output :

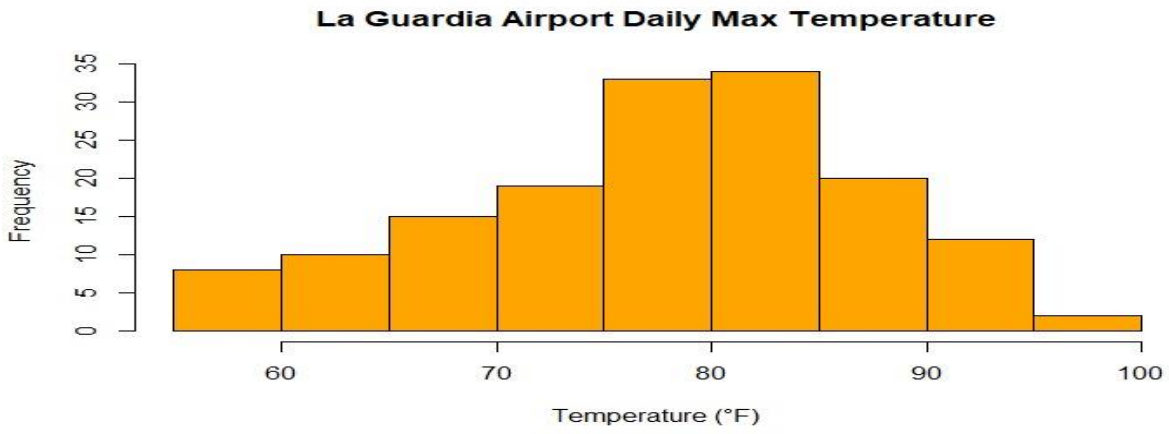
```
Ozone Solar.R Wind Temp Month Day
1    41     190  7.4   67     5   1
2    36     118  8.0   72     5   2
3    12     149 12.6   74     5   3
4    18     313 11.5   62     5   4
5    NA      NA 14.3   56     5   5
6    28      NA 14.9   66     5   6
```

Program :

```
# Create a histogram for daily maximum temperature (Temp)
hist(airquality$Temp,
     main = "La Guardia Airport Daily Max Temperature",
     xlab = "Temperature (°F)",
```

```
    ylab = "Frequency",
    col = "orange",
    border = "black",
    breaks = 10) # Adjust 'breaks' to change bin count
```

Output :



WEEK - 2

EXP NO. : 2

Aim : Load Air Passengers dataset in R and visualize the data using line chart that shows increase in air passengers over given time period.

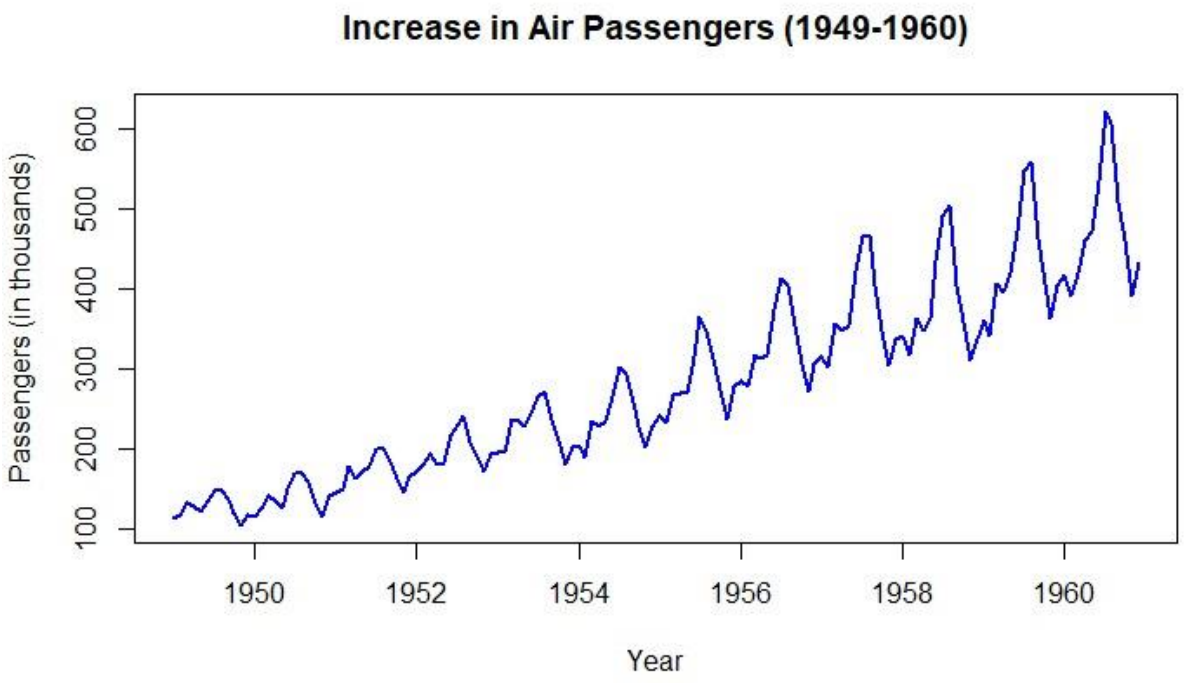
Explanation:

- The primary purpose of this program is to conduct Time Series Analysis to identify the long-term trend and seasonal patterns in air travel.
- By plotting the AirPassengers dataset as a line chart, the program achieves several key goals:
- **Trend Identification**: It visually confirms the steady upward growth of the airline industry from 1949 to 1960.
- **Seasonality Detection:** It reveals the cyclical spikes (summer peaks) and dips (winter lows) that repeat every 12 months.
- **Variance Analysis:** It shows how the fluctuations grow larger as the total number of passengers increases (known as a multiplicative time series).
- **Forecasting Base:** It provides the essential visual foundation needed to build mathematical models for predicting future passenger traffic.
- In short, it transforms a list of 144 monthly data points into a clear "story" of industry expansion and consumer travel habits.

Program :

```
data("AirPassengers")


# Create a line chart showing the increase over time
plot(AirPassengers,
    main = "Increase in Air Passengers (1949-1960)",
    xlab = "Year",
    ylab = "Passengers (in thousands)",
    col = "blue",
    lwd = 2) # lwd sets line width
```

Output :


Increase in Air Passengers (1949-1960)
Passengers (in thousands)
100
200
300
400
500
600
1950
1952
1954
1956
1958
1960
Year

WEEK - 3

EXP NO. : 3A

Aim : Load iris dataset in R, visualize the data using different Bar Charts and also demonstrate the use of stacked plots.

Explanation :

- The Iris dataset is one of the most famous datasets in statistics and machine learning.
- Introduced by British biologist Ronald Fisher in 1936, it contains 150 records of three different species of iris flowers: Setosa, Versicolor, and Virginica.
- Each record includes four physical measurements (in centimetres): Sepal Length, Sepal Width, Petal Length, and Petal Width.

The primary purpose of this program is Comparative Profile Analysis. It transforms individual flower measurements into a high-level summary to achieve the following:

- Species Comparison: It allows you to instantly see which species is generally larger (e.g., Virginica) and which is significantly smaller (e.g., Setosa).
- Visualizing Composition (Stacked Plots): The stacked plot shows the "total profile" of each flower by stacking its four measurements. This helps identify which specific trait (like Petal Length) contributes most to the difference between species.
- Data Simplification: It condenses 150 rows of raw numbers into a single, easy-to-interpret visual. This is essential for Exploratory Data Analysis (EDA) before building classification models.
- Identifying Key Differentiators: By looking at the segments, it becomes clear that Petal Length is the most distinctive feature for telling these three species apart.

Program :

```
# To download from the internet (Run once only)

install.packages("ggplot2", repos = "https://cloud.r-project.org")

install.packages("tidyr", repos = "https://cloud.r-project.org")


# To prepare the software for use

library(ggplot2)

library(tidyr)


# Loading the 'iris' data

data(iris)
```

```
# Displays the first 6 rows
head(iris)
```

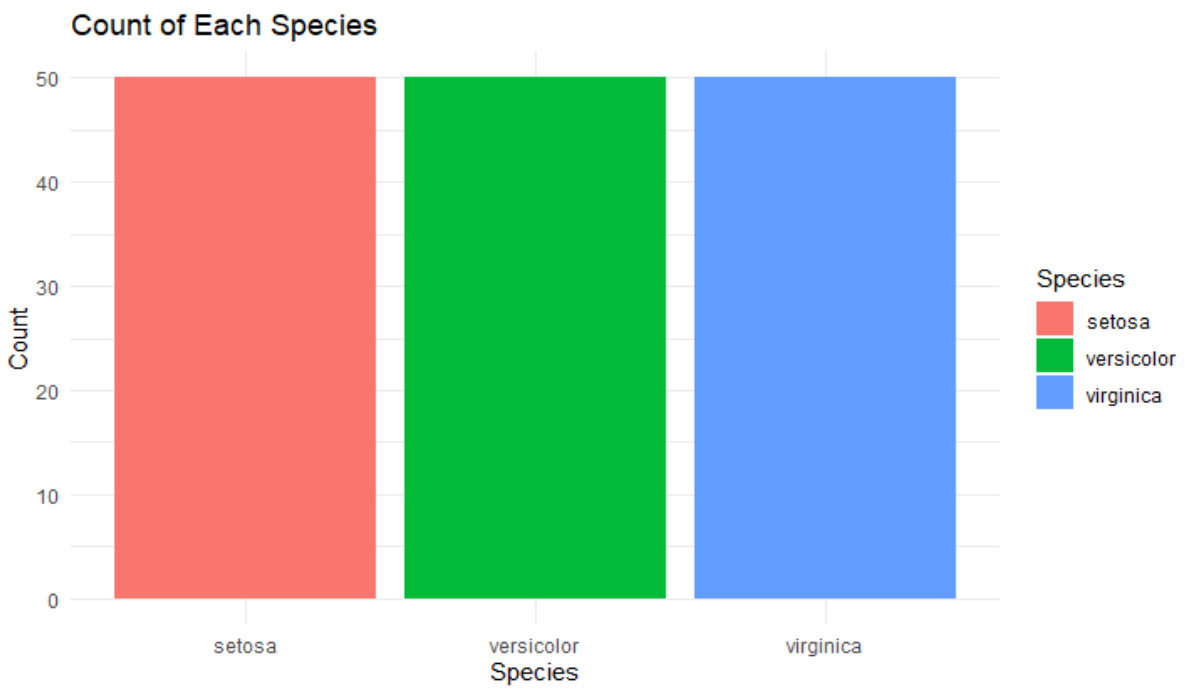
Output :

```
Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
```

Program :

```
ggplot(iris, aes(x = Species, fill = Species)) +
  geom_bar() +
  labs(title = "Count of Each Species", y = "Count") +
  theme_minimal()
```

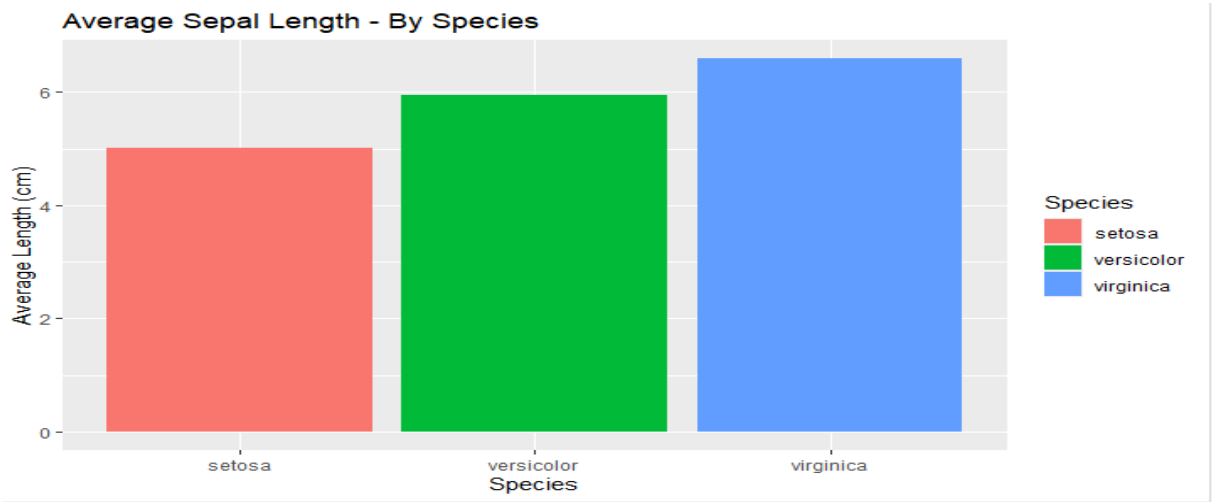
Output :



Program :

```
iris_means <- aggregate(. ~ Species, data = iris, mean) # Calculating the mean


ggplot(iris_means, aes(x = Species, y = Sepal.Length, fill = Species)) +
  geom_bar(stat = "identity") +
  labs(title = "Average Sepal Length - By Species", y = "Average Length (cm)")
```

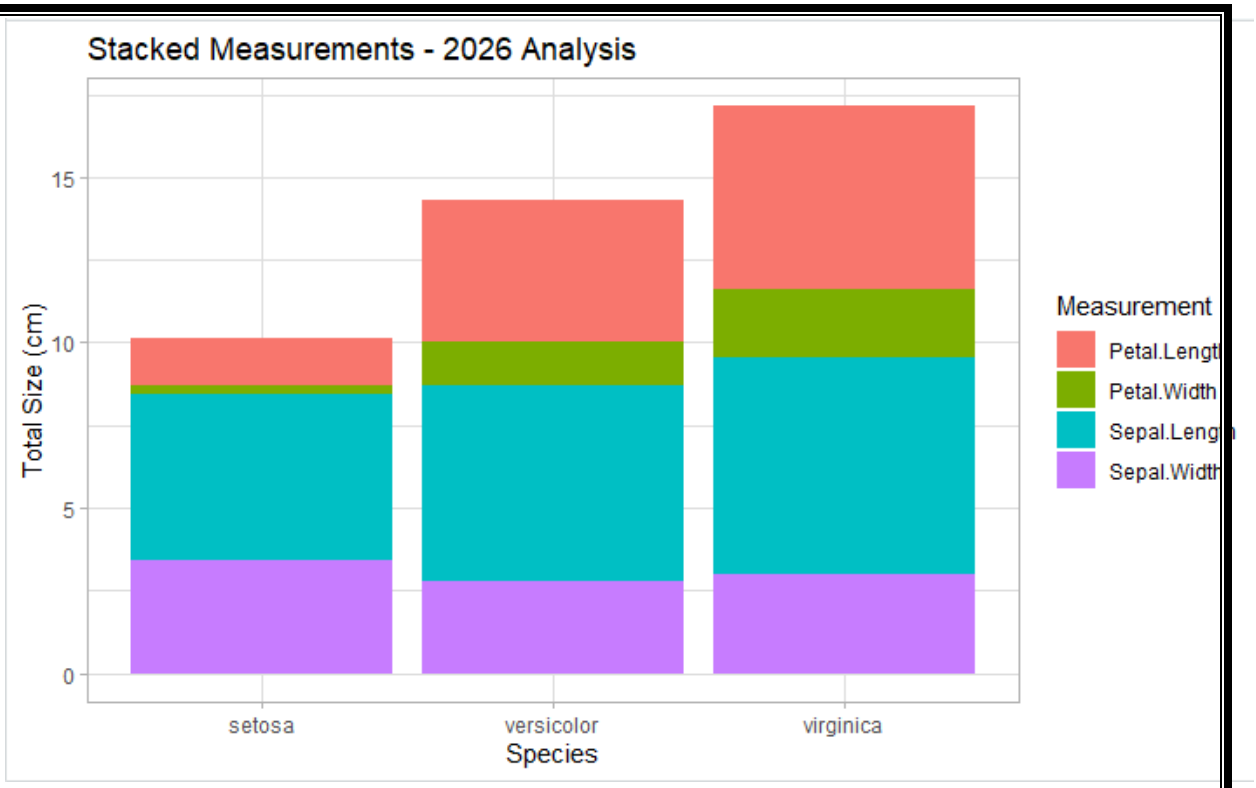
Output :



Program :

```
# Converting data to Long format
iris_long <- pivot_longer(iris, cols = 1:4, names_to = "Measurement", values_to = "Value")


# Calculating the mean for the transformed data
iris_long_means <- aggregate(Value ~ Species + Measurement, data = iris_long, mean)


# Creating the Stacked Plot
ggplot(iris_long_means, aes(x = Species, y = Value, fill = Measurement)) +
  geom_bar(stat = "identity", position = "stack") +
  labs(title = "Stacked Measurements - 2026 Analysis", y = "Total Size (cm)") +
  theme_light()
```

Output :


Stacked Measurements - 2026 Analysis
Total Size (cm)
15
10
5
0
setosa
versicolor
virginica
Species
Measurement
Petal.Lengt
Petal.Width
Sepal.Leng
Sepal.Width

EXP NO. : 3B

Aim : Load air quality dataset in R and visualize ozone concentration in air.

Explanation:

- The airquality dataset is a built-in R data frame containing daily air quality measurements from New York City between May and September 1973. It consists of 153 observations on 6 variables:

Ozone: Mean ozone in parts per billion (ppb).
Solar.R: Solar radiation.
Wind: Average wind speed.
Temp: Maximum daily temperature.
Month/Day: The date of the measurement.

The primary purpose is Environmental Data Analysis to understand pollution patterns:

- Assessing Air Safety: The histogram shows how often ozone levels exceeded "safe" thresholds during the 1973 summer.
- Identifying Data Gaps: This dataset contains NA (missing values) in the Ozone column; the program helps identify if the missing data affects our understanding of the distribution.
- Detecting Skewness: Ozone data is often "right-skewed" (most days have low pollution, but a few days have extremely high, dangerous peaks). This visualization makes those extreme events obvious.
- Correlation Context: It provides a baseline for further study, such as seeing if high temperatures (Temp) directly lead to higher Ozone levels.

Program :

```
data("airquality")

# Check the first few rows to understand the structure

head(airquality)
```

Output :

```
Ozone Solar.R Wind Temp Month Day

1    41     190  7.4   67     5   1

2    36     118  8.0   72     5   2

3    12     149 12.6   74     5   3

4    18     313 11.5   62     5   4
```

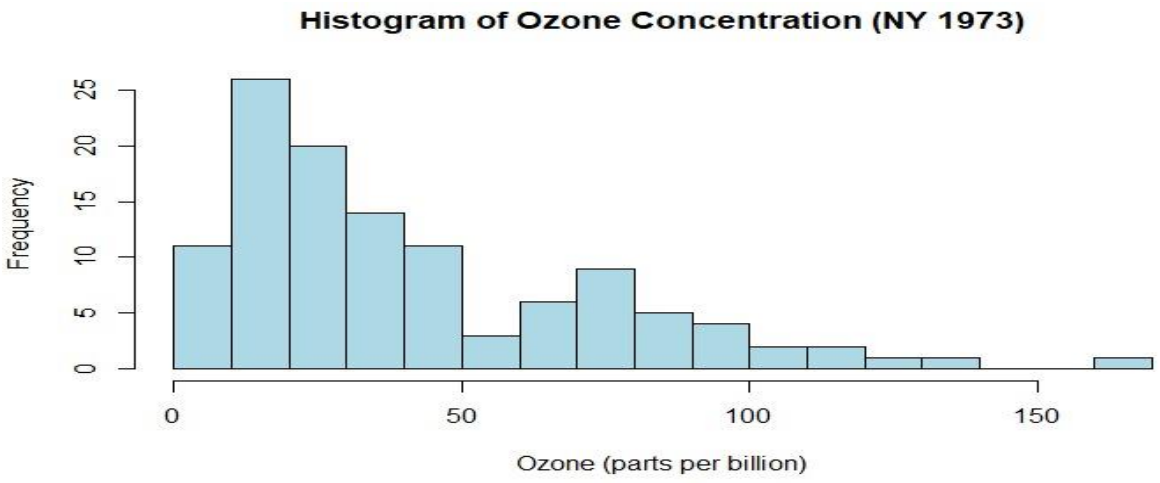
5 NA NA 14.3 56 5 5

6 28 NA 14.9 66 5 6

Program :

```
hist(airquality$Ozone,
    main = "Histogram of Ozone Concentration (NY 1973)",
    xlab = "Ozone (parts per billion)",
    col = "lightblue",
    border = "black",
    breaks = 15)
```

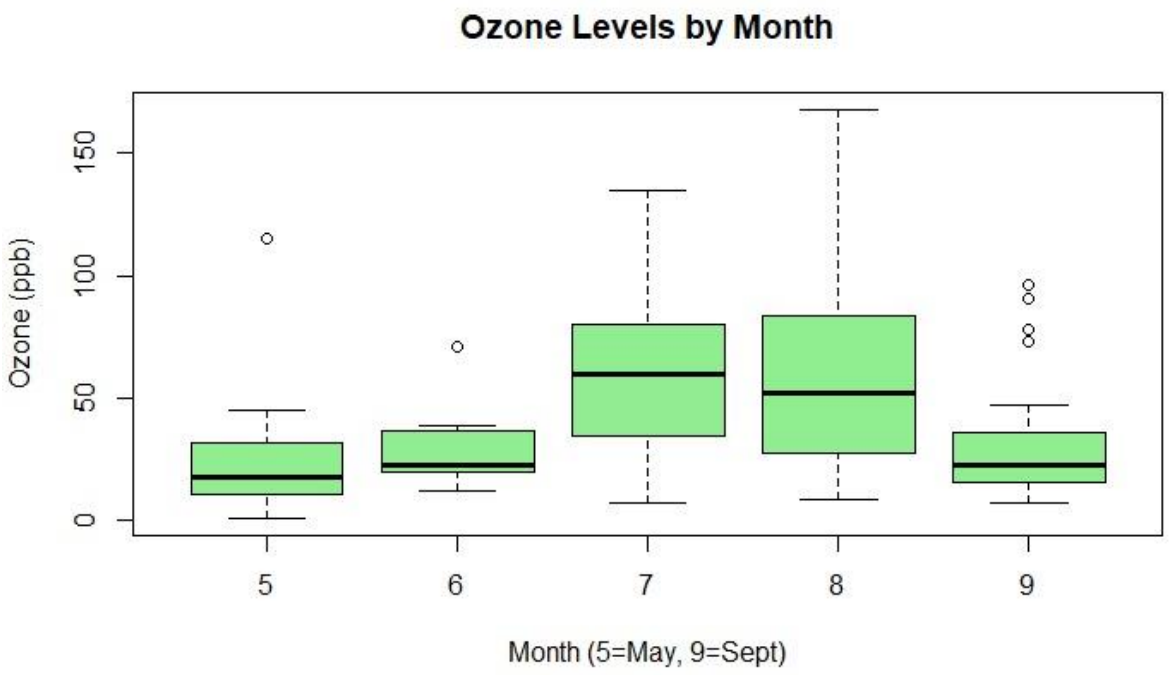
Output :



Program :

```
boxplot(Ozone ~ Month,
        data = airquality,
        main = "Ozone Levels by Month",
        xlab = "Month (5=May, 9=Sept)",
        ylab = "Ozone (ppb)",
        col = "lightgreen")
```

Output :


Ozone Levels by Month
Ozone (ppb)
0
50
100
150
5
6
7
8
9
Month (5=May, 9=Sept)

WEEK - 4

EXP NO. : 4A

Aim : Load iris dataset in R, visualize the data using different Box plots including group by option and also use color palette to represent species.

Explanation:

- The Iris dataset is a classic "multivariate" dataset introduced by Ronald Fisher in 1936. It contains 150 samples from three species of Iris flowers (Setosa, Versicolor, and Virginica). For each flower, four features were measured:

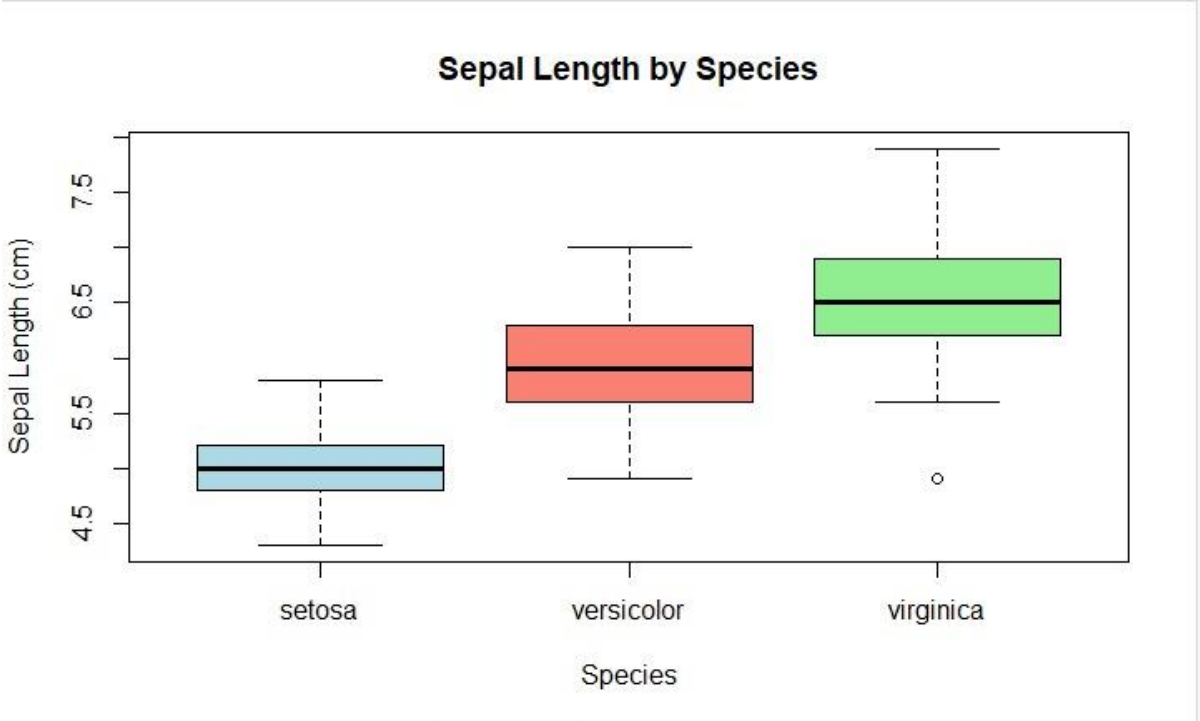
1. Sepal Length
2. Sepal Width
3. Petal Length
4. Petal Width

- The primary purpose of using grouped box plots on this dataset is Statistical Distribution Analysis:
- Comparison of Medians: It allows you to see at a glance that Setosa has significantly shorter petals than the other two species.
- Visualizing Spread (Variation): The "height" of the box shows how much the measurements vary within a single species.
- Identifying Overlap: It helps determine if the species are distinct enough to be separated by a machine learning model. For example, you'll notice Setosa is completely isolated, while Versicolor and Virginica have some overlap.
- Outlier Detection: The "whiskers" and individual dots reveal if any specific flowers have unusually large or small measurements compared to their peers.

Program :

```
data(iris)

head(iris)

boxplot(Sepal.Length ~ Species,

        data = iris,

        main = "Sepal Length by Species",

        xlab = "Species",

        ylab = "Sepal Length (cm)",

        col = c("lightblue", "salmon", "lightgreen"))
```

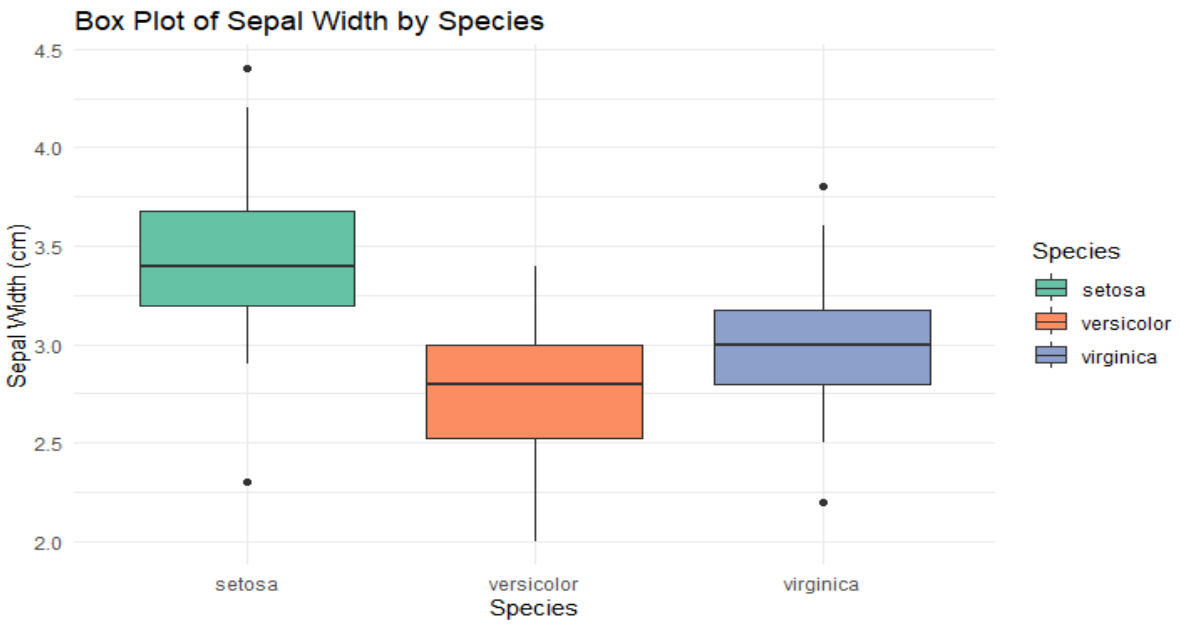
Output :



Program :

```
options(download.file.method = "wininet")
install.packages("ggplot2")
capabilities("libcurl")
install.packages("ggplot2")
library(ggplot2)
ggplot(iris, aes(x = Species, y = Sepal.Width, fill = Species)) +
  geom_boxplot() +
  scale_fill_brewer(palette = "Set2") +
  labs(title = "Box Plot of Sepal Width by Species",
       x = "Species",
       y = "Sepal Width (cm)") +
  theme_minimal()
```

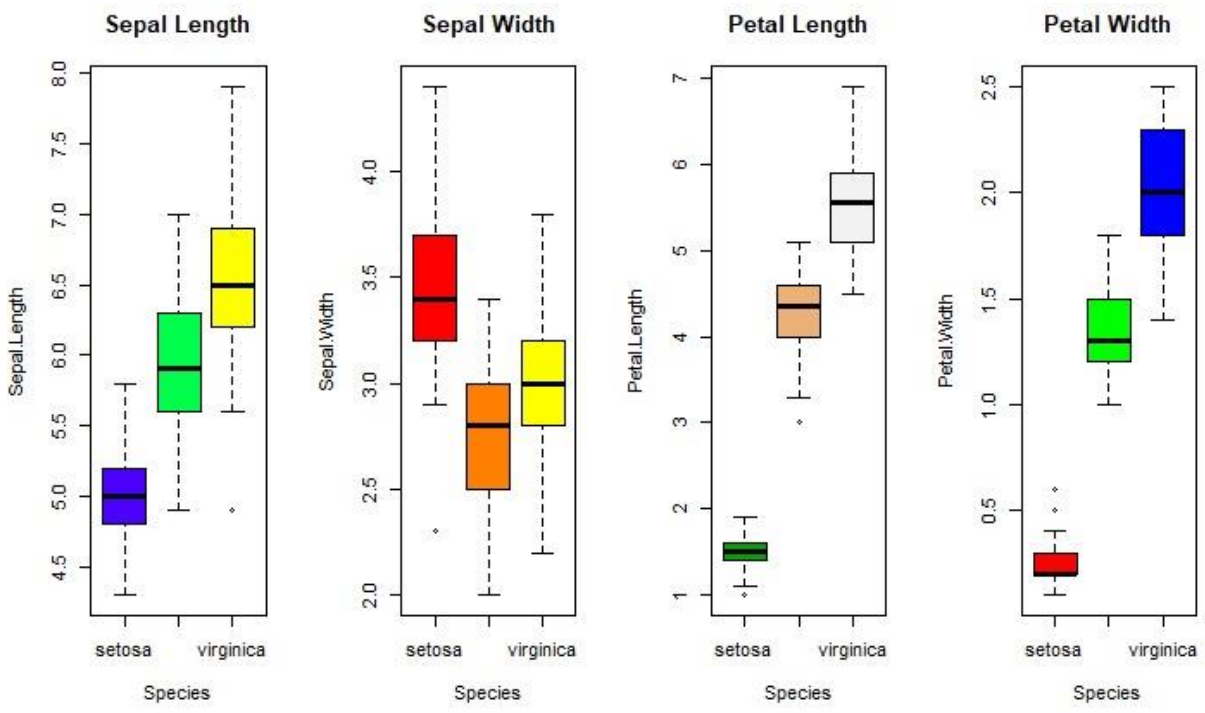
Output :



Program :

```
par(mfrow = c(1, 4))
boxplot(Sepal.Length ~ Species, data = iris, col = topo.colors(3), main = "Sepal Length")
boxplot(Sepal.Width ~ Species, data = iris, col = heat.colors(3), main = "Sepal Width")
boxplot(Petal.Length ~ Species, data = iris, col = terrain.colors(3), main = "Petal Length")
boxplot(Petal.Width ~ Species, data = iris, col = rainbow(3), main = "Petal Width")
```

Output :



EXP NO. : 4B

Aim : Load air quality dataset in R and visualize air quality parameters using box plots.

Explanation:

- The airquality dataset is a built-in R data frame containing 153 daily air quality measurements from May to September 1973 in New York City. It tracks four key environmental variables:

Ozone: Mean ozone in parts per billion.

Solar.R: Solar radiation in Langleys.

Wind: Average wind speed in miles per hour.

Temp: Maximum daily temperature in degrees Fahrenheit.

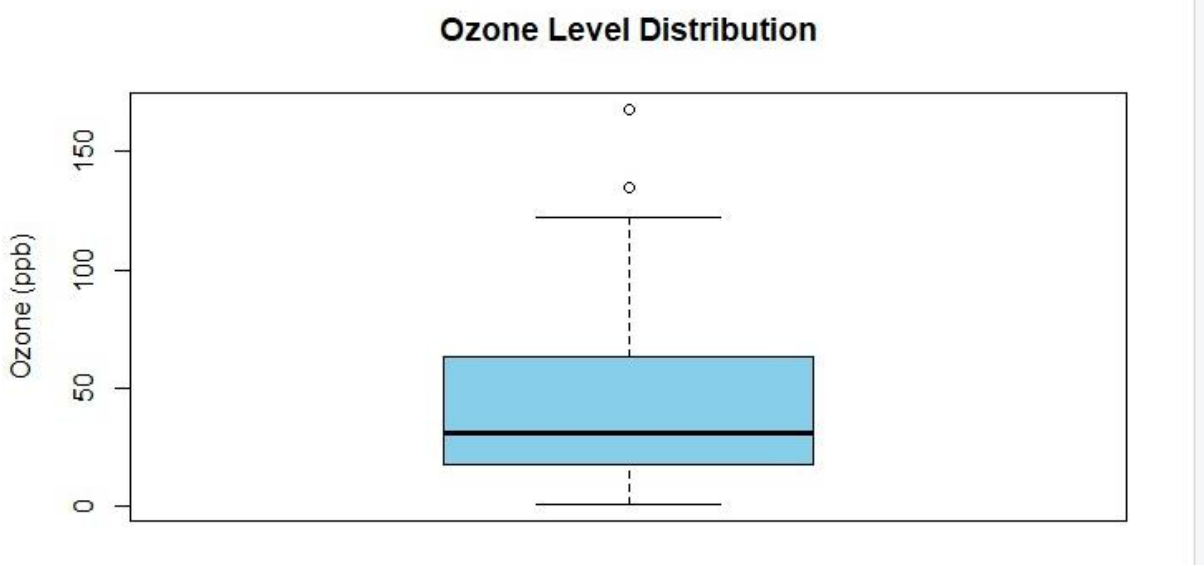
The primary purpose is Statistical Distribution Comparison:

- **Visualizing Spread and Range:** The "boxes" show where the middle 50% of the data lies (Interquartile Range), helping you see if measurements like Wind are consistent or highly variable.
- **Identifying Outliers:** Box plots use "whiskers" and individual dots to highlight extreme pollution days (unusually high Ozone levels) that might be dangerous to public health.
- **Detecting Skewness:** By looking at the median line inside the box, you can tell if the data is symmetrical. For instance, Ozone is often right-skewed, meaning most days have low pollution, but a few days have very high peaks.
- **Data Cleaning:** It helps identify Missing Values (NAs). If a box plot doesn't appear for a variable, it often signals that the data needs cleaning or that NAs are present.

Program :

```
data("airquality")

head(airquality)

boxplot(airquality$Ozone, main="Ozone Level Distribution", col="skyblue", ylab="Ozone (ppb)")

boxplot(airquality[, 1:4], main="Air Quality Parameters", col=c("red", "blue", "green", "yellow"))

boxplot(Temp ~ Month, data=airquality, main="Temperature by Month", xlab="Month", ylab="Temp (F)", col="orange")
```

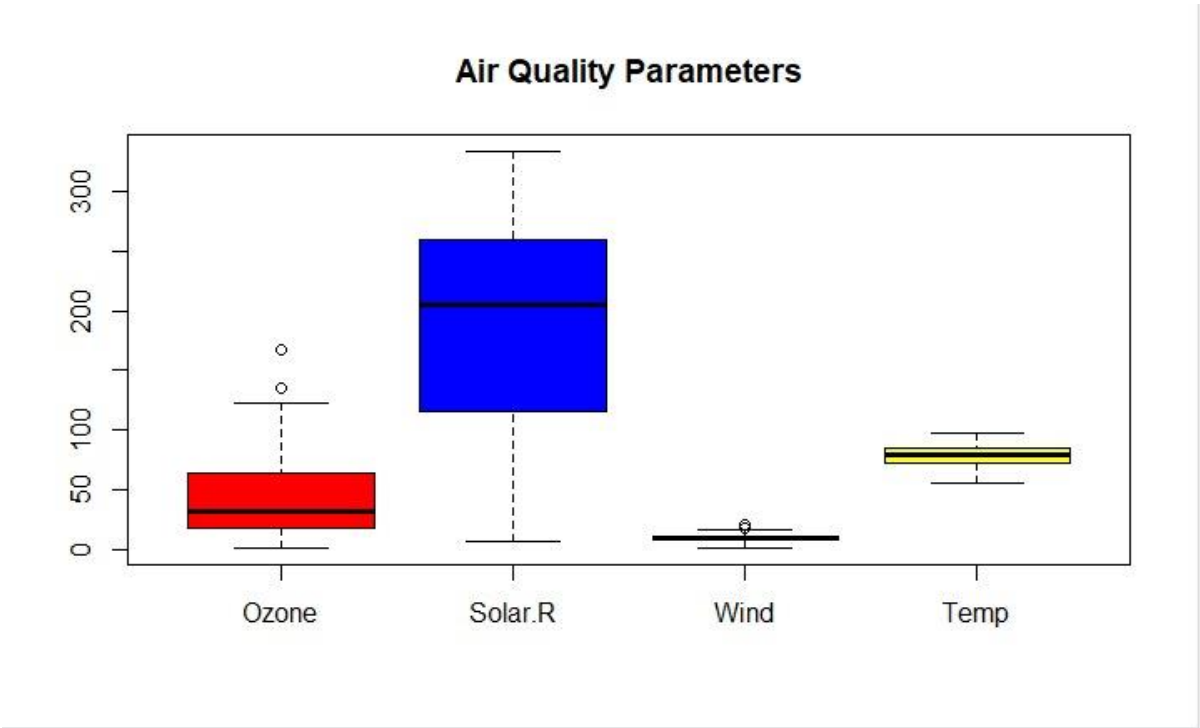
Output :



Program :

boxplot(airquality[, 1:4], main="Air Quality Parameters", col=c("red", "blue", "green", "yellow"))

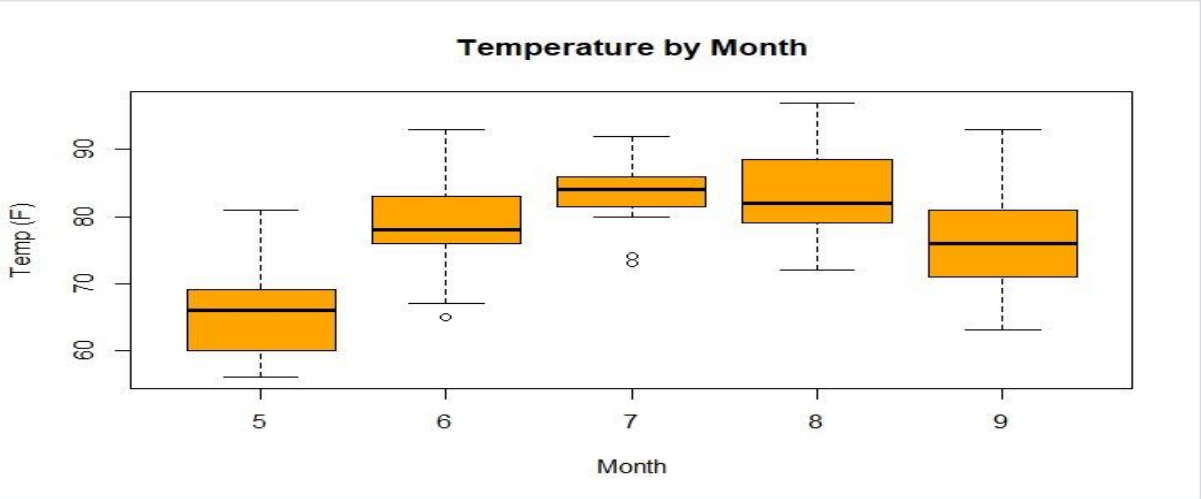
Output :



Program :

```
boxplot(Temp ~ Month, data=airquality, main="Temperature by Month", xlab="Month",
ylab="Temp (F)", col="orange")
```

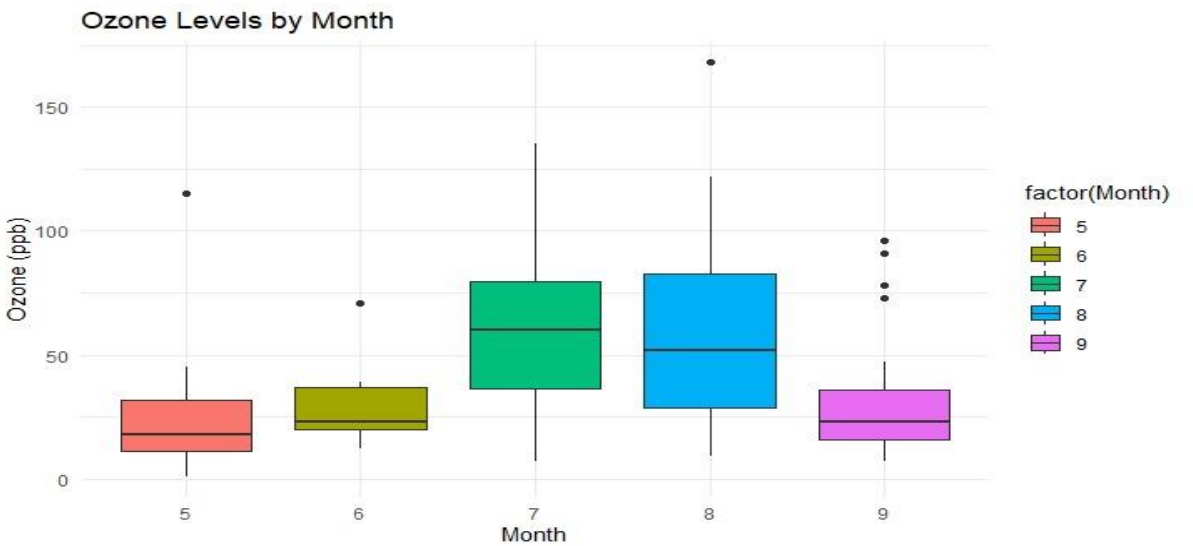
Output :



Program :

```
library(ggplot2)
ggplot(airquality, aes(x = factor(Month), y = Ozone, fill = factor(Month))) +
  geom_boxplot() +
  labs(title = "Ozone Levels by Month", x = "Month", y = "Ozone (ppb)") +
  theme_minimal()
```

Output :



WEEK - 5

EXP NO. : 5

Aim : Visualize iris dataset using simple scatter, multivariate scatter plot and also visualize scatter plot matrix to visualize multiple variables across each other.

Explanation:

- The Iris dataset is a multivariate data collection consisting of 150 samples from three species of flowers (Setosa, Versicolor, and Virginica). It records four features: Sepal Length, Sepal Width, Petal Length, and Petal Width. It is the "gold standard" dataset for testing classification algorithms.

The primary purpose of this program is Correlation and Cluster Analysis:

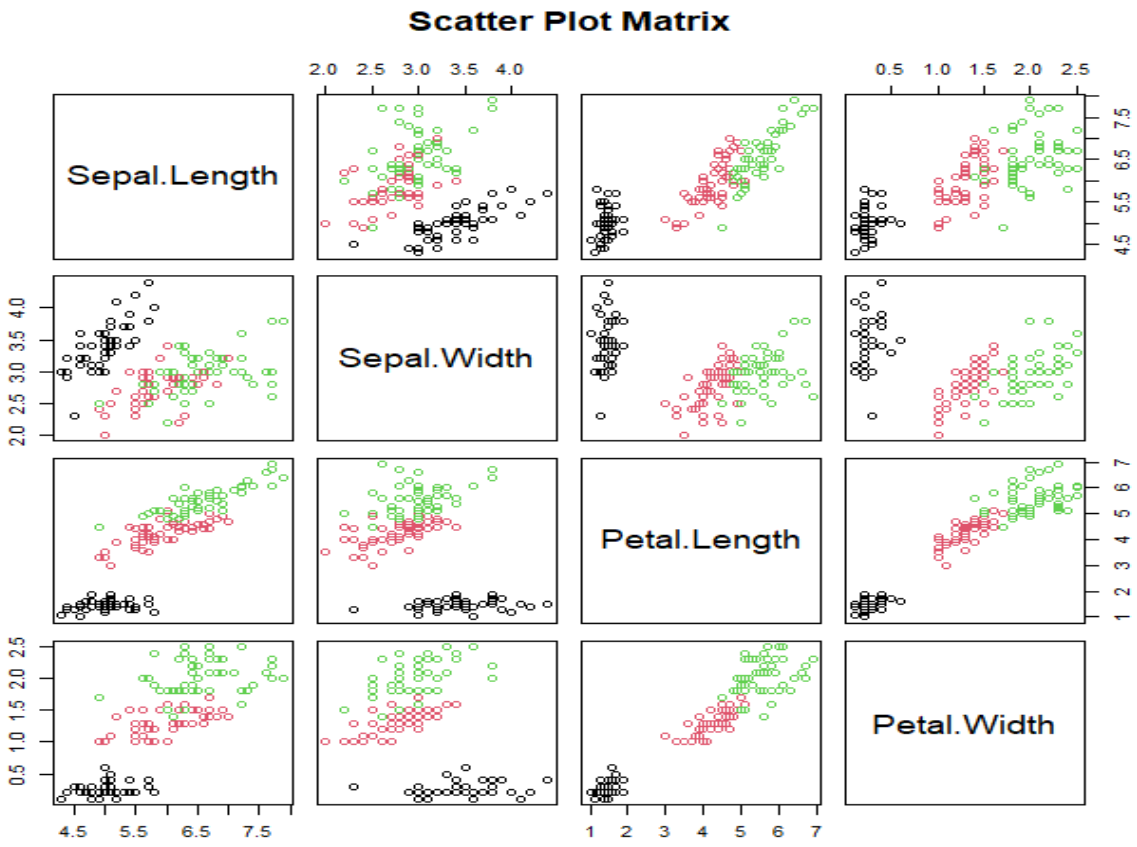
- **Identifying Relationships:** The simple scatter plot shows if variables move together (e.g., as Petal Length increases, Petal Width usually increases, showing a strong positive correlation).
- **Species Separation (Clustering):** The multivariate plot reveals how different species group together in "clusters." It visually proves that Setosa is easily separable from the others, while Versicolor and Virginica are more closely related.
- **High-Dimensional Overview:** The Scatter Plot Matrix (pairs) allows you to see every possible relationship in the dataset at once. This helps identify which specific pair of variables (like Petal Length vs. Petal Width) is the best predictor for identifying a species.
- **Feature Selection:** By seeing which variables create the clearest "gaps" between species, researchers can decide which data points are most important for building machine learning models.

Program :

```
# Load ggplot2 library

library(ggplot2)

# Create a scatter plot

ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width)) +

  geom_point() +

  labs(title = "Simple Scatter Plot: Sepal Dimensions",

       x = "Sepal Length",

       y = "Sepal Width")

# Load ggplot2 library

library(ggplot2)
```

```
# Create a multivariate scatter plot
ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width, color = Species)) +
  geom_point() +
  labs(title = "Multivariate Scatter Plot by Species",
       x = "Sepal Length",
       y = "Sepal Width",
       color = "Species")
# Scatter Plot Matrix for iris dataset
pairs(iris[, 1:4],
      col = as.numeric(iris$Species),
      main = "Scatter Plot Matrix")
```

Output :



WEEK – 6

EXP NO. : 6

AIM : Load diamonds dataset in R and visualize the structure in datasets with large data points using hexagon binning and also add color palette.

Explanation:

- The diamonds dataset is a collection of prices and attributes for nearly 54,000 round-cut diamonds. It includes 10 variables, most notably the "Four Cs" of diamond quality:

Carat: The weight/size of the diamond.

Cut: Quality of the cut (Fair, Good, Very Good, Premium, Ideal).

Color: Diamond colour, from J (worst) to D (best).

Clarity: A measurement of how clear the diamond is.

Price: The price in US dollars.

Dimensions: The x, y, and z physical measurements.

The primary purpose of this program is Density Visualization in Large Data:

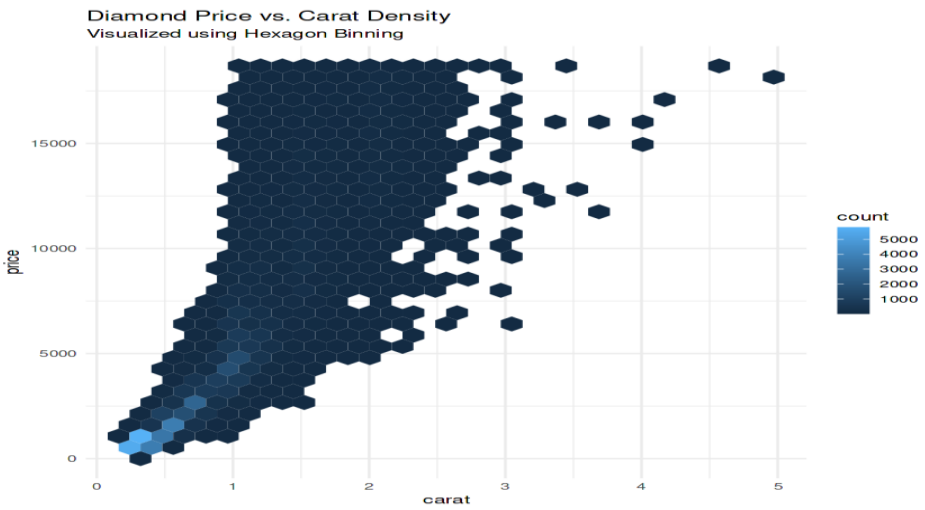
- **Solving Overplotting:** In a standard scatter plot with 54,000 points, the dots would overlap so much that the graph would just look like a solid black blob.
- **Visualizing Data Concentration:** Hexagon binning divides the plot into hexagonal "bins" and colors them based on how many data points fall inside each. This allows you to see the "hot spots" where most diamonds are concentrated (e.g., many diamonds are small and inexpensive).
- **Identifying Trends:** By using a color palette (like the gradient from blue to red), you can instantly see the relationship between weight and price while also understanding the volume of data at every point.
- **Market Analysis:** It helps researchers understand the distribution of the diamond market—for instance, showing that while prices generally rise with carats, there is a huge concentration of diamonds at specific "round number" carat weights.

Program :

```
# Install packages if you haven't already

# install.packages(c("ggplot2", "hexbin", "RColorBrewer"))


library(ggplot2)

library(hexbin)

library(RColorBrewer) # Used for advanced color palettes
```

```
# Load the dataset
data(diamonds)
#price to the y-axis, which are the most common variables for analyzing diamond value.
# Basic Hexbin plot
ggplot(diamonds, aes(x = carat, y = price)) +
  geom_hex() +
  theme_minimal() +
  labs(title = "Diamond Price vs. Carat Density",
       subtitle = "Visualized using Hexagon Binning")
```

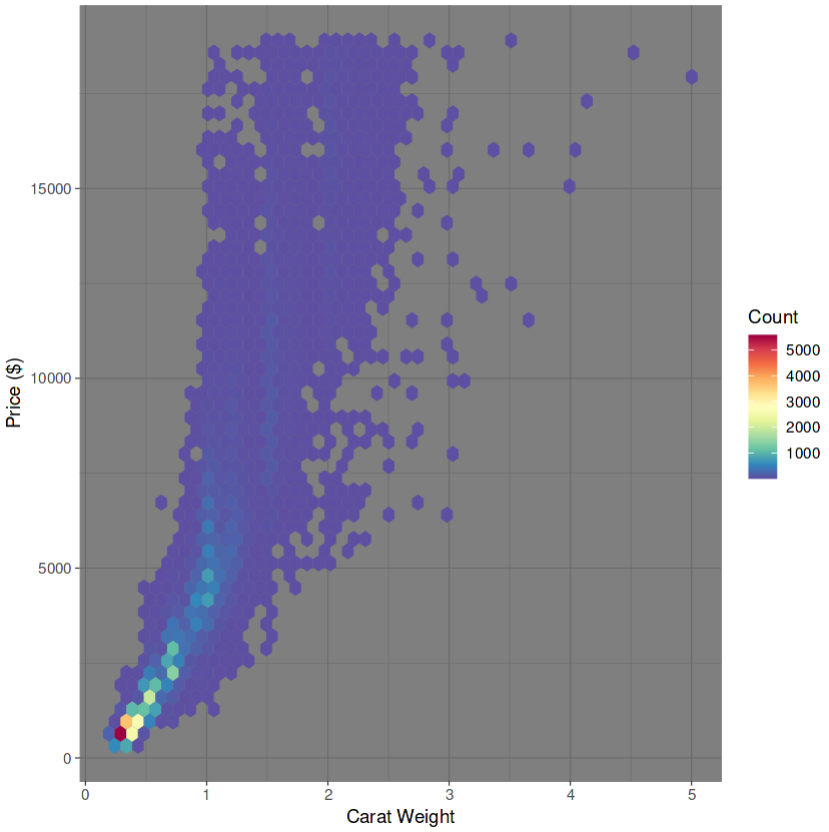
Output :



Program :

```
# Creating a custom color palette using RColorBrewer
# 'Spectral' or 'RdYlBu' are great for showing density gradients
my_colors <- colorRampPalette(rev(brewer.pal(11, "Spectral")))

ggplot(diamonds, aes(x = carat, y = price)) +
  geom_hex(bins = 50) + # Increase bins for more detail
  scale_fill_gradientn(colors = my_colors(100)) +
  theme_dark() + # Dark theme makes colors pop
  labs(fill = "Count", x = "Carat Weight", y = "Price ($)")
```

Output :


Price ($)
15000
10000
5000
0
0
1
2
3
4
5
Carat Weight
Count
5000
4000
3000
2000
1000

WEEK – 7

EXP NO.: 7

Aim : Load Hair Eye Color dataset in R and plot categorical data using mosaic plot.

Explanation:

The HairEyeColor dataset is a built-in R table that contains the distribution of hair color, eye color, and sex for 592 statistics students at the University of Delaware. It tracks:

Hair Color: Black, Brown, Red, Blond.

Eye Color: Brown, Blue, Hazel, Green.

Sex: Male, Female.

The primary purpose is Association Analysis for Categorical Data:

- Visualizing Proportions: Unlike a bar chart, the area of the tiles in a mosaic plot is proportional to the number of people in that category. You can instantly see which combination (e.g., Brown Hair/Brown Eyes) is the most common.
- Identifying Correlations: It helps you see if certain traits "stick together." For example, the plot will show that "Blond" hair has a much larger "Blue" eye segment compared to "Black" hair.
- Detecting Independence: It allows researchers to visually check if hair color and eye color are independent or if knowing one helps you predict the other.
- Multi-dimensional Summary: It condenses a complex 3-way table (Hair x Eye x Sex) into a single visual that is easier to explain to a non-technical audience than a table of raw numbers.

Program :

```
data("HairEyeColor")


# View the structure (it's a 3D table: Hair x Eye x Sex)

print(HairEyeColor)
```

Output :

```
, , Sex = Male

      Eye

Hair    Brown Blue Hazel Green

  Black    32   11    10     3

  Brown    53   50    25    15

  Red      10   10     7     7
```

Blond 3 30 5 8

, , Sex = Female

Eye

Hair Brown Blue Hazel Green

Black 36 9 5 2

Brown 66 34 29 14

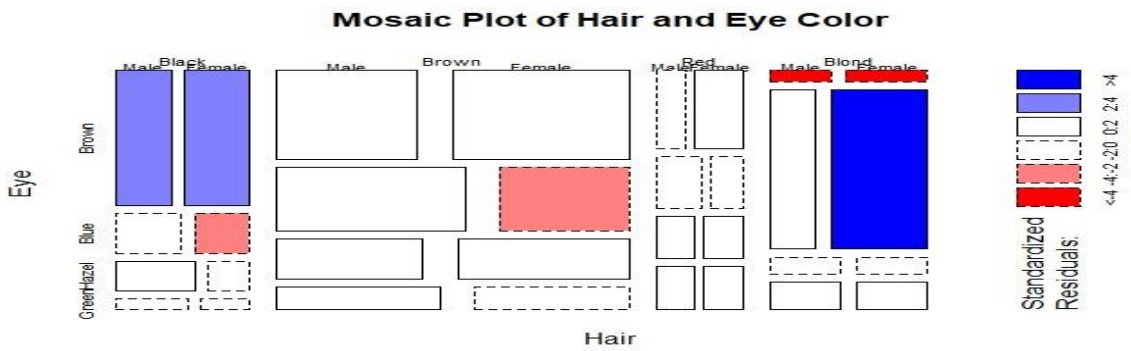
Red 16 7 7 7

Blond 4 64 5 8

Program :

```
mosaicplot(HairEyeColor, shade = TRUE, main = "Mosaic Plot of Hair and Eye Color")
```

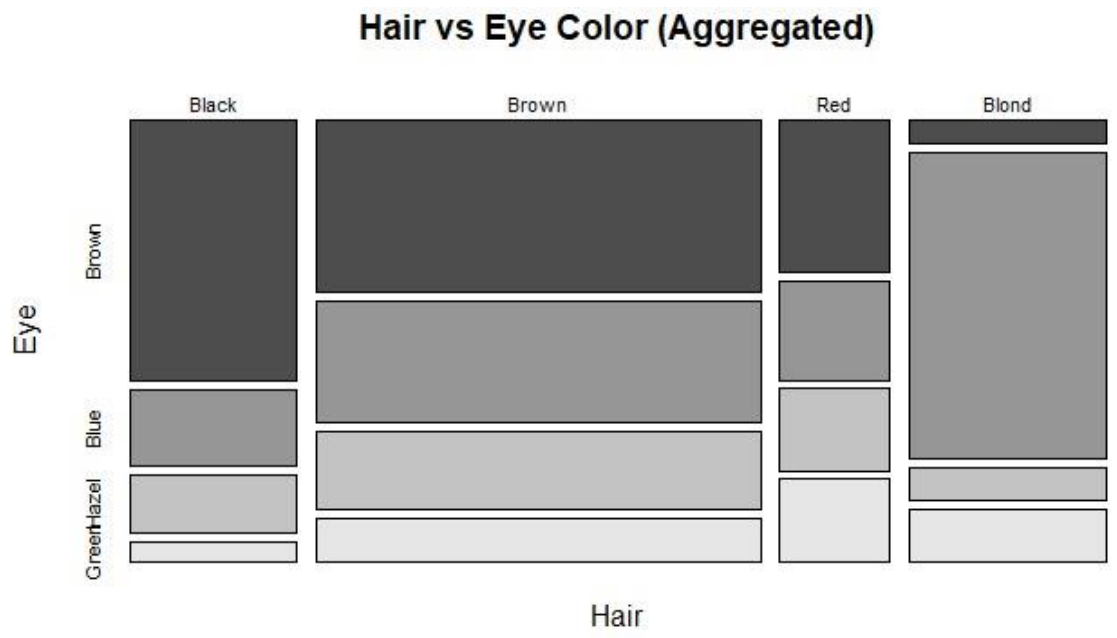
Output :



Program :

```
hair_eye_sum <- apply(HairEyeColor, c(1, 2), sum)
mosaicplot(hair_eye_sum, color = TRUE, main = "Hair vs Eye Color (Aggregated)")
```

Output :


Hair vs Eye Color (Aggregated)
Black
Brown
Red
Blond
Brown
Blue
Green
Hazel
Eye
Hair

WEEK – 8

EXP NO. : 8

Aim : Load mtcars dataset in R and visualize data using heat map.

Explanation:

The Motor Trend Car Road Tests (mtcars) dataset comprises fuel consumption and 10 aspects of automobile design and performance for 32 automobiles (1973–74 models). It includes 11 variables:

- mpg: Miles per gallon (Fuel efficiency).
- cyl: Number of cylinders.
- disp: Displacement (cu.in.).
- hp: Gross horsepower.
- drat: Rear axle ratio.
- wt: Weight (1000 lbs).
- qsec: 1/4 mile time (Acceleration).
- vs: Engine (0 = V-shaped, 1 = straight).
- am: Transmission (0 = automatic, 1 = manual).
- gear: Number of forward gears.
- carb: Number of carburetors.

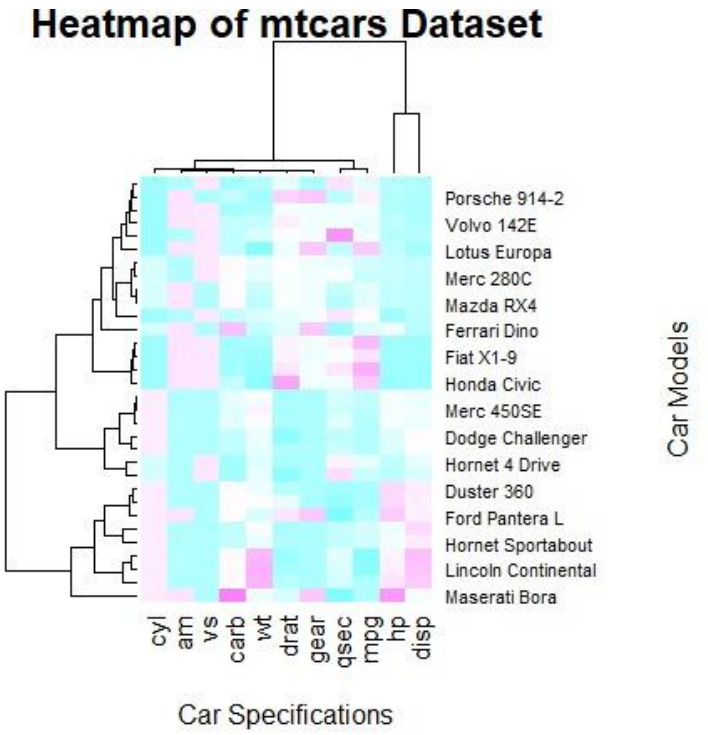
The primary purpose is Pattern Recognition across Multiple Variables:

- **Identifying Correlations:** You can quickly see which variables "behave" similarly. For example, cars with high hp (horsepower) and cyl (cylinders) usually show a darker/different color for low mpg (efficiency).
- **Comparative Benchmarking:** It allows you to see the "profile" of a specific car model. You can instantly spot which cars are "heavy outliers" versus which are "high-performance" models.
- **Visual Summarization:** Instead of looking at a table of 352 numbers (32 rows x 11 columns), the heat map uses color intensity to show where the highest and lowest values lie across the entire fleet.
- **Cluster Discovery:** If you enable "clustering" (the default trees on the side of a heatmap), the program will automatically group cars that have similar mechanical blueprints.

Program :

```
data(mtcars)

# Convert to a numeric matrix (required for heatmap function)
mtcars_matrix <- as.matrix(mtcars)

heatmap(mtcars_matrix,
        scale = "column",
        col = cm.colors(256),
        main = "Heatmap of mtcars Dataset",
        xlab = "Car Specifications",
        ylab = "Car Models",
        margins = c(5, 10))
```

Output :



WEEK – 9

EXP NO. : 9

Aim : Install leaflet library in R and perform different map visualizations.

Explanation:

- Leaflet is one of the most popular open-source JavaScript libraries for interactive maps. The R package allows you to control those JavaScript maps directly from your R console. Unlike static plots, Leaflet maps allow users to:

1. Zoom in and out.
2. Pan across different regions.
3. Click on markers for pop-ups and more information.
4. Layer different types of map data (satellite views, street maps, terrain).

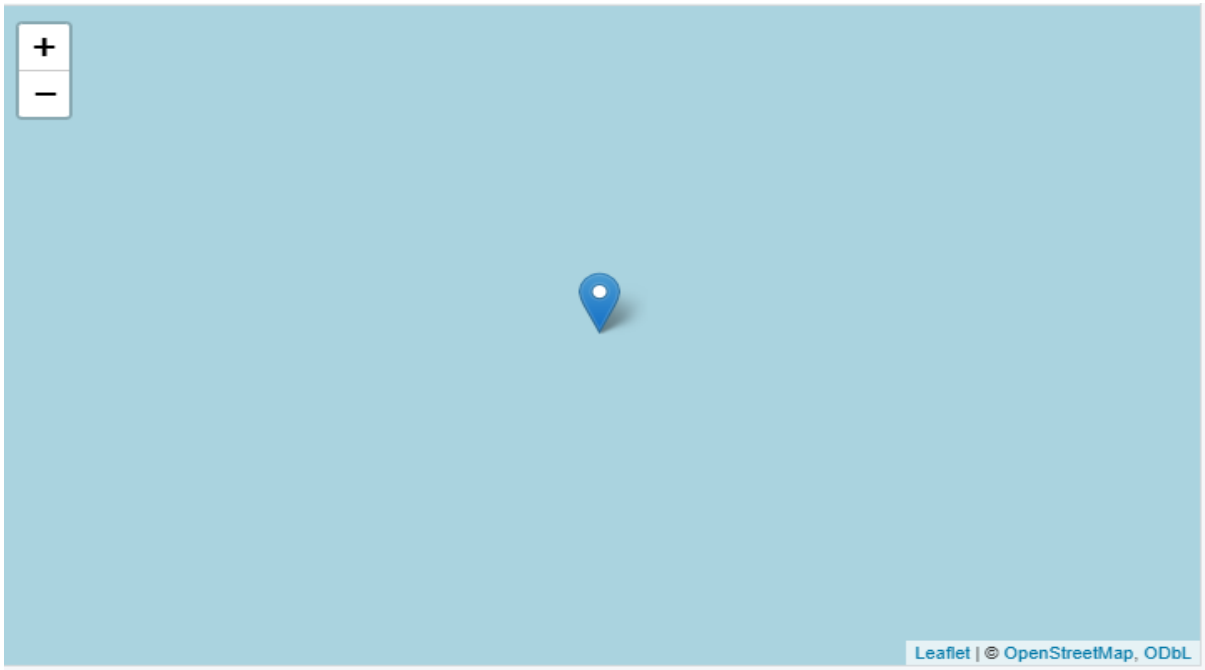
The primary purpose is Geospatial Data Visualization:

- **Interactive Exploration:** Instead of a flat image, it provides a "living" map that the user can interact with to find specific details at a local level.
- **Mapping Coordinates:** It translates raw Latitude and Longitude data into physical locations on a map, making geographical data (like store locations, accident sites, or weather stations) easy to understand.
- **Visualizing Density (Heatmaps):** It is often used to show "hotspots" (e.g., areas with high crime or high population density) by layering colors over specific regions.
- **Spatial Relationships:** It helps researchers see how data points are clustered geographically, which is essential for urban planning, logistics, and environmental tracking.

Program :

```
# Install using the global cloud mirror to avoid the error

install.packages("leaflet", repos = "https://cloud.r-project.org")

# Now load it up!

library(leaflet)

leaflet() %>%

  addTiles() %>%

  addMarkers(lng = 0, lat = 0, popup = "The Center of the World?")
```

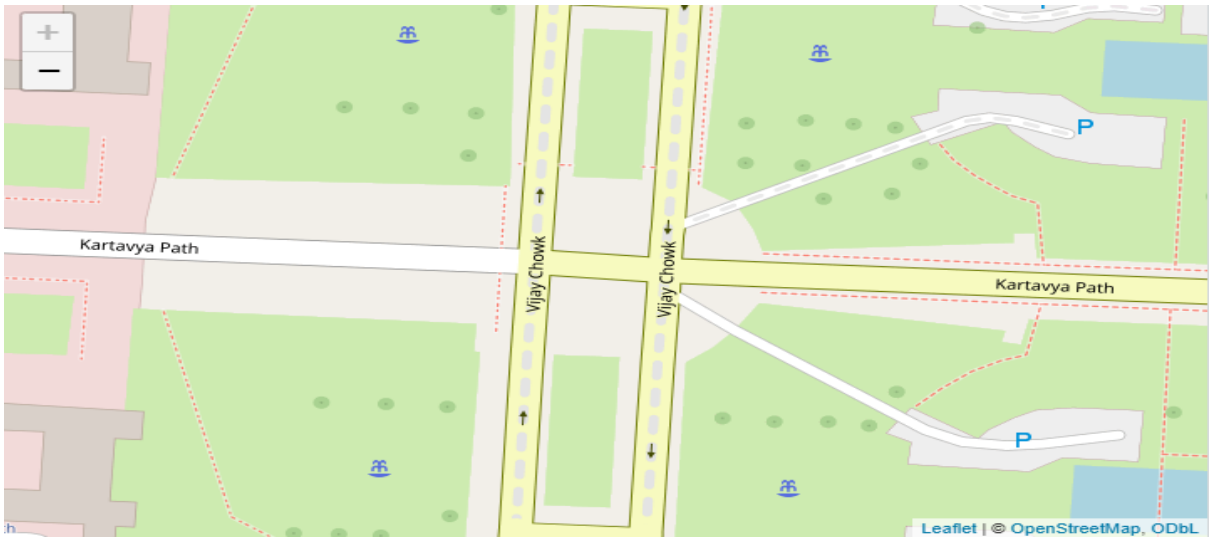
Output :



Program :

```
leaflet() %>%
  addTiles() %>%
  setView(lng = 77.2090, lat = 28.6139, zoom = 25) # Centers on New Delhi
```

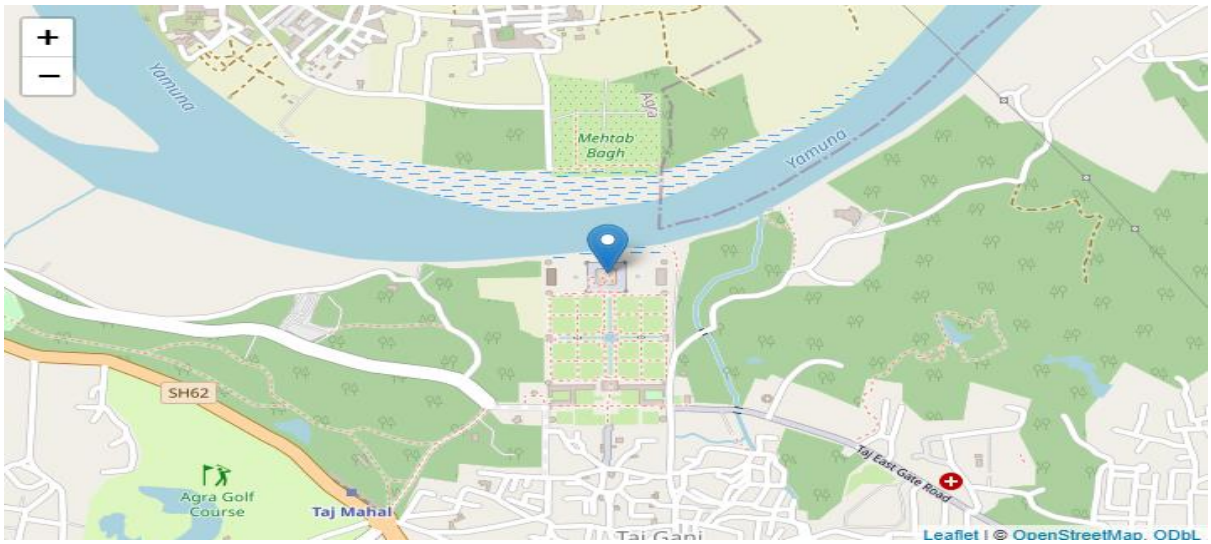
Output :



Program :

```
leaflet() %>%
  addTiles() %>%
  addMarkers(lng = 78.0421, lat = 27.1751, popup = "The Taj Mahal")
```

Output :



Program :

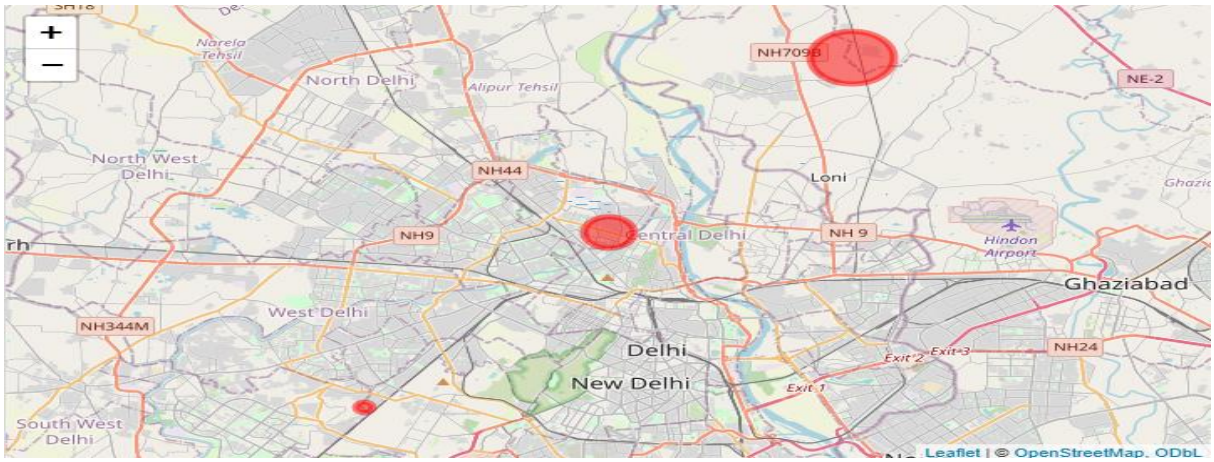
```
leaflet() %>%
  addProviderTiles(providers$Esri.WorldImagery) %>% # Satellite view
  addMarkers(lng = -74.0445, lat = 40.6892, popup = "Statue of Liberty")
```

Output :

Program :

```
df <- data.frame(lat = c(28.6, 28.7, 28.8), lng = c(77.1, 77.2, 77.3), mag = c(5, 15, 25))
leaflet(df) %>%
  addTiles() %>%
  addCircleMarkers(radius = ~mag, color = "red", fillOpacity = 0.5)
```

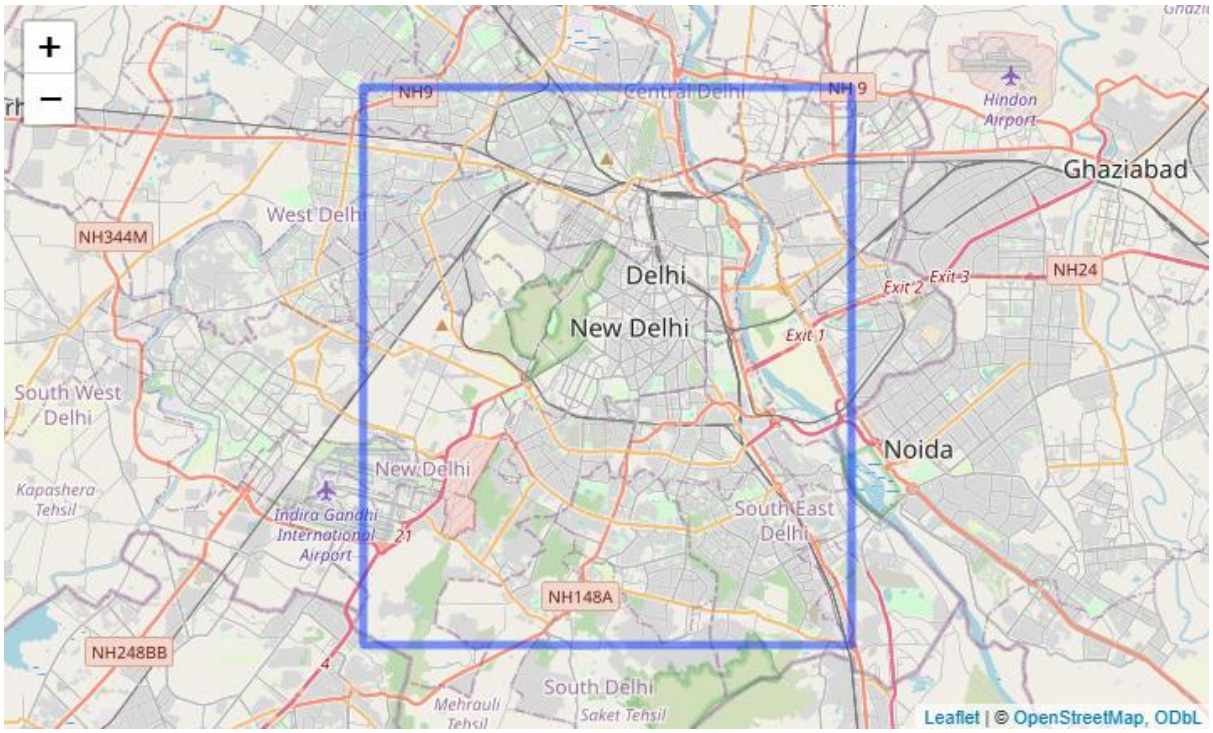
Output :



Program :

```
leaflet() %>%
  addTiles() %>%
  addRectangles(
    lng1 = 77.1, lat1 = 28.5,
    lng2 = 77.3, lat2 = 28.7,
    fillColor = "transparent"
  )
```

Output :


NH9
Central Delhi
NH 9
Hindon Airport
Ghaziabad
West Delhi
NH344M
Delhi
New Delhi
Exit 2
Exit 3
NH24
Exit 1
South West Delhi
Noida
New Delhi
Kapashera Tehsil
Indira Gandhi International Airport
South East Delhi
NH148A
NH248BB
South Delhi
Mehrauli Tehsil
Saket Tehsil
Leaflet | © OpenStreetMap, ODbL

WEEK – 10

EXP NO. : 10

Aim : Visualize iris dataset using 3d graphs such as scatter3d, cloud, xyplot.

Explanation:

In a 2D world, we can only see how two traits (like Length vs Width) relate. In a 3D context, the Iris dataset becomes a "cloud" of points in space. Each species occupies a specific spatial volume. For example, you can see that Setosa is a tight, isolated cluster in one corner of the 3D box, while Versicolor and Virginica form two elongated, nearly touching "clouds."

The purpose is High-Dimensional Cluster Analysis:

True Separation: Some species look overlapped in 2D. Adding a third dimension (Z-axis) often reveals a "gap" between groups that wasn't visible before, proving they are distinct.

Structural Understanding: It helps researchers see the "shape" of the data. Is the species cluster a sphere, a line, or a flat plane? This tells us how the plant's traits grow in relation to each other.

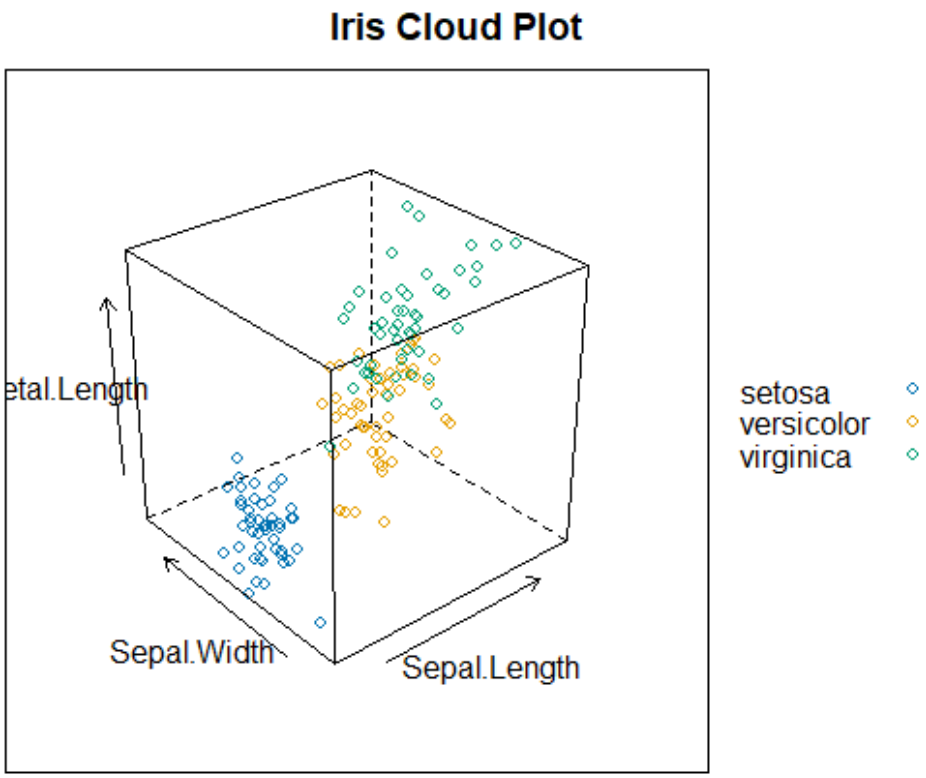
Advanced Pattern Recognition: 3D plots make it easier to spot multivariate outliers—flowers that might look normal in 2D but are clearly "floating" away from the group in 3D space.

Feature Interaction: It demonstrates how three variables (like Sepal Length, Width, and Petal Length) combine to create the unique "signature" of a species.

Program :

```
library(lattice)


# Create a 3D cloud plot

cloud(Petal.Length ~ Sepal.Length * Sepal.Width,

    data = iris,

    groups = Species,

    auto.key = TRUE, # Adds a legend

    main = "Iris Cloud Plot")
```

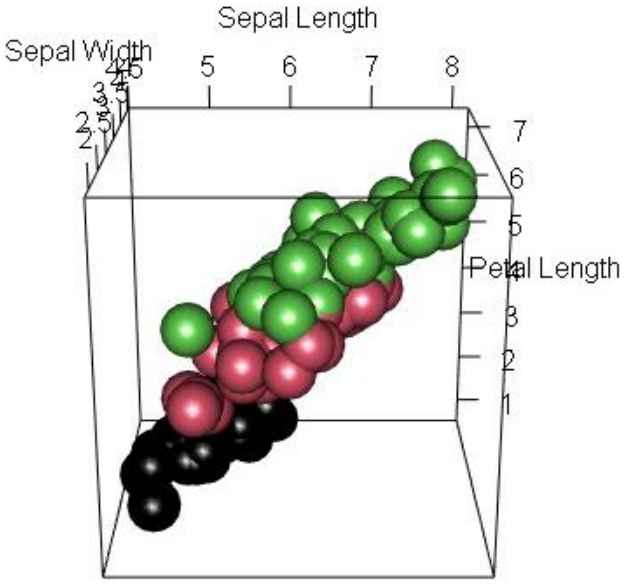
Output :



Program :

```
install.packages("rgl")
library(rgl)

# Interactive 3D plot
plot3d(iris$Sepal.Length, iris$Sepal.Width, iris$Petal.Length,
       col = as.numeric(iris$Species),
       size = 5,
       type = 's', # 's' for spheres, 'p' for points
       xlab = "Sepal Length", ylab = "Sepal Width", zlab = "Petal Length")
```

Output :



WEEK – 11

EXP NO. : 11

Aim : Make use of correlogram to visualize data in correlation matrices for iris dataset.

Explanation:

A correlogram (or correlation matrix plot) is a graphical tool used to display the relationship between multiple variables simultaneously.

Colours: Usually, blue represents a positive correlation, red represents a negative correlation, and white means no relationship.

Intensity/Size: Darker colours or larger shapes indicate a stronger relationship (closer to +1 or -1).

Shapes: Using ellipses shows the "tightness" of the relationship; a thin diagonal ellipse means a very strong correlation.

The primary purpose is Feature Relationship Analysis:

Spotting Redundancy: If two variables (like Petal Length and Petal Width) show a dark blue, nearly perfect correlation, it tells the researcher they are redundant—you might only need one of them for a machine learning model.

Multivariate Overview: Instead of looking at six different scatter plots, a correlogram lets you see every possible pair-wise relationship in one clean, color-coded grid.

Pattern Discovery: It highlights which physical traits of the Iris flower "grow together." For example, it visually proves that as the petals get longer, they almost always get wider.

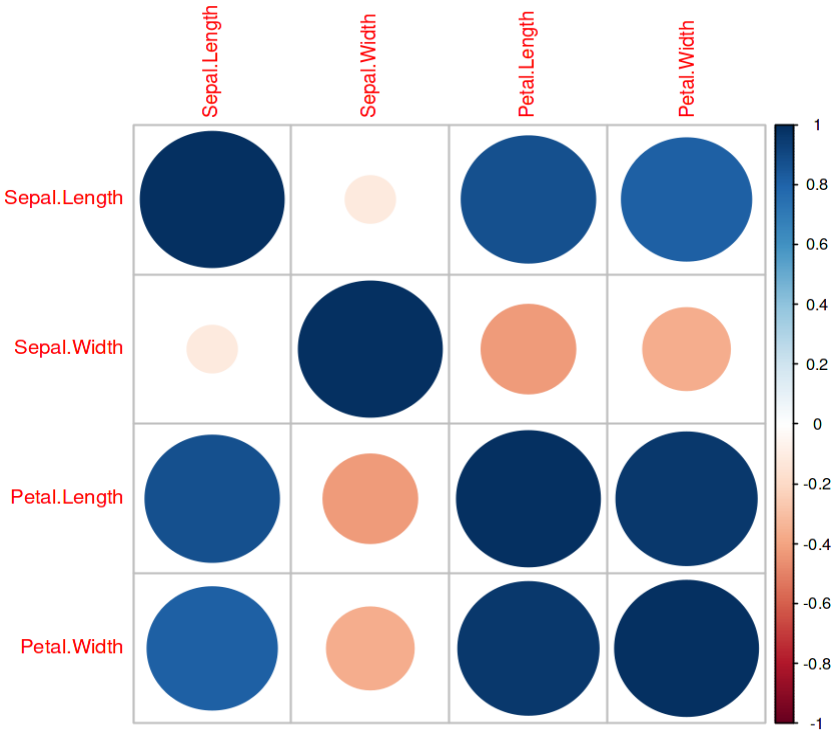
Variable Selection: It helps identify which features have the weakest relationships (the "faded" or white cells), which is useful for simplifying complex data.

Program :

```
install.packages("corrplot")
library(corrplot)


# Install the package if you haven't already

 # install.packages("corrplot")

 library(corrplot)

 # Load the built-in dataset

 data(iris)


 # Calculate the correlation matrix for the first 4 numeric columns

 iris_cor <- cor(iris[, 1:4])
```

```
corrplot(iris_cor, method = "circle")
```

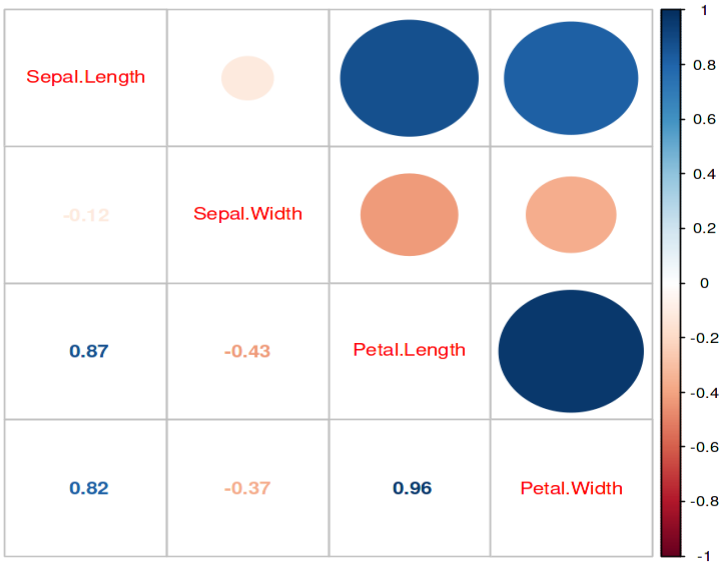
Output :



Program :

```
corrplot.mixed(iris_cor, lower = "number", upper = "circle")
```

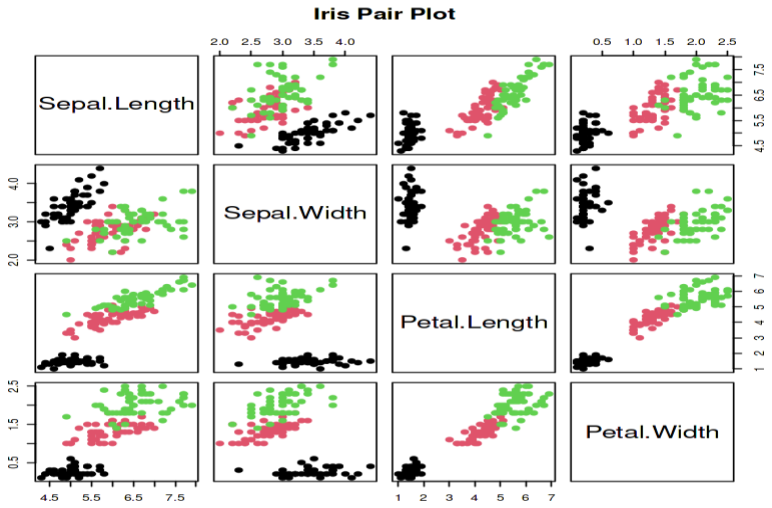
Output :



Program :

```
# Visualise pairwise relationships colored by species

pairs(iris[, 1:4], col = iris$Species, pch = 19, main = "Iris Pair Plot")
```

Output :



WEEK – 12

EXP NO. : 12

Aim : Install maps library in R and draw different map visualizations.

Explanation:

The maps library is a classic R package used for displaying geographical maps. It contains a database of coordinates for the world, US states, and US counties, allowing users to generate static maps without needing external shapefiles or complex GIS software. It is often used as a base layer for more advanced packages like ggplot2 or ggmap.

The primary purpose of map visualization programs is Spatial Data Representation:

Geographic Context: It provides a physical layout for trends and patterns that are otherwise just numbers (e.g., showing where disease outbreaks or sales are concentrated).

Data Summarization: It groups complex datasets by region (countries or states) to reveal how data varies across different areas.

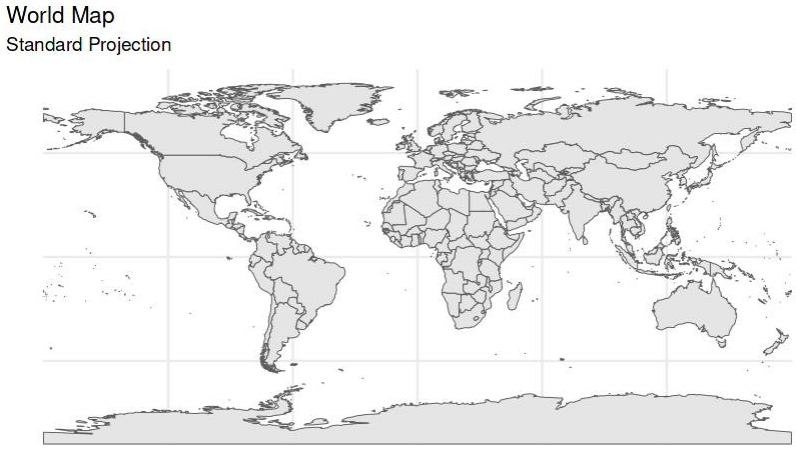
Pattern Recognition: It helps researchers identify clusters, outliers, and spatial relationships between variables.

Decision Making: Maps serve as effective tools for visualising socio-cultural or economic phenomena to aid in assessment and evaluation.

Program :

```
install.packages(c("ggplot2", "sf", "rnaturalearth", "rnaturalearthdata"))

library(ggplot2)
library(sf)
library(rnaturalearth)
# Get world map data
world <- ne_countries(scale = "medium", returnclass = "sf")

# Basic plot
ggplot(data = world) +
  geom_sf() +
  theme_minimal() +
  labs(title = "World Map", subtitle = "Standard Projection")
```

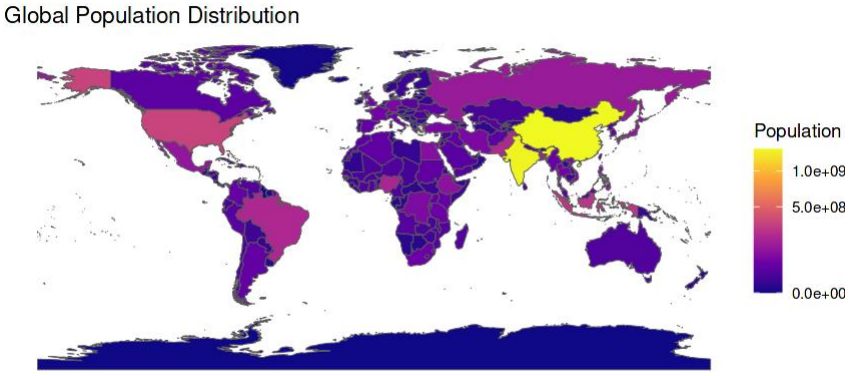
Output :



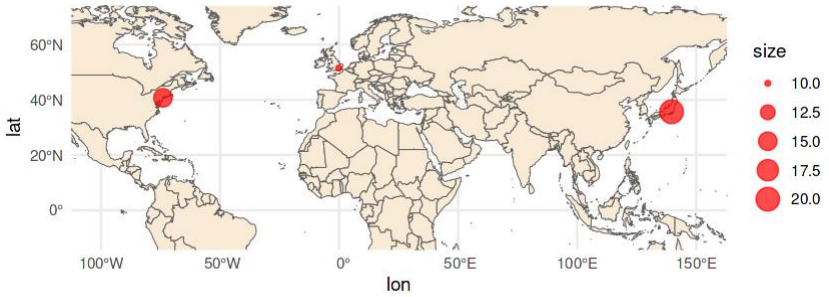
Program :

```
ggplot(data = world) +
  geom_sf(aes(fill = pop_est)) + # Fill based on population estimate
  scale_fill_viridis_c(option = "plasma", trans = "sqrt") + # Better color scale
  theme_void() +
  labs(fill = "Population", title = "Global Population Distribution")
```

Output :



Program :

```
# Sample data for cities
cities <- data.frame(
  name = c("London", "New York", "Tokyo"),
  lat = c(51.5074, 40.7128, 35.6762),
  lon = c(-0.1278, -74.0060, 139.6503),
  size = c(10, 15, 20)
)

ggplot(data = world) +
   geom_sf(fill = "antiquewhite") +
   geom_point(data = cities, aes(x = lon, y = lat, size = size),
              color = "red", alpha = 0.7) +
   coord_sf(xlim = c(-100, 150), ylim = c(-10, 70)) + # Zoom into specific area
   theme_minimal()
```

Output :